Lab 3: Memory Allocator

CSCI 3334 · UTRGV Fall 2026

Overview

Weight: 10% | **Out:** Mon Oct 12 | **Due:** Mon Nov 2

You will write `malloc` and `free`. Your allocator gets a heap, one long run of bytes, and carves it into blocks: it hands blocks out when the program asks for memory, takes them back when the program frees them, and reuses them for later requests. This is the allocator from Lecture 11, written for real and checked on every request.

Your code goes in one file, `mm.c`, as `mm_malloc`, `mm_free`, `mm_realloc` and `mm_calloc`. A test program we give you, the **driver**, runs lists of requests against your functions and checks every answer: that each block is aligned, inside the heap, does not overlap another block, and still holds the data the driver wrote into it.

The lab is graded in five levels:

1. **Level 1 (60 pts):** `mm_malloc` and `mm_free` with an implicit free list and first fit. This is the code on the Lecture 11 slides. Level 1 on its own is a passing grade.
2. **Level 2 (15 pts):** split big free blocks, and merge (coalesce) neighbours when a block is freed.
3. **Level 3 (10 pts):** `mm_realloc` and `mm_calloc`.
4. **Level 4 (10 pts):** an explicit free list, so finding a free block does not mean walking past every allocated one.
5. **Level 5 (5 pts):** a heap checker, `mm_check`, and a clean build.

There is also a **+10 bonus** for using memory well: growing a block in place in `mm_realloc`, and choosing free blocks better than first fit does.

> **Important**
>
> **Checkpoint: Mon Oct 26**
>
> Show me `make test L=1` passing at least the first check, `t1-lecture.tr`, which is the Lecture 11 example below. The checkpoint is graded pass/fail on effort, not correctness, and counts toward participation. It is there so you get help early if you are stuck.

The midterm is on Wed Oct 14, two days after the lab goes out. Level 1 is about 20 lines of C, so you can have it working before the midterm.

Setup

Download `lab3.zip` from the Lab 3 assignment on **Brightspace** and unzip it. It unpacks to a `lab3/` folder:

```
unzip lab3.zip
cd lab3
make            # builds ./mdriver from mm.c and the driver
make test       # checks every level
make test L=1   # checks only level 1
./mdriver -v traces/t1-lecture.tr     # runs one trace and shows each request
make leaks      # memory errors (valgrind) on Linux and WSL, leaks on macOS
```

This lab is plain C. It builds and runs on macOS as well as on Linux and WSL.

File	What it is
`mm.c`	your allocator. The block layout, helper functions and `mm_init` are written for you. You fill in the functions marked `YOUR CODE HERE`. **This is the only file you submit.**
`mm.h`	the functions you write. Do not change it.
`memlib.c`, `memlib.h`	the simulated heap and `mem_sbrk`. Do not change them.
`mdriver.c`	the driver: runs a trace and checks every request. Do not change it.
`traces/`	the request lists (traces) the tests use. Each one starts with a comment that says what it tests.
`tests/run_tests.sh`	the test script behind `make test`

Warning

Rules for mm.c

- Get all of your memory from `mem_sbrk`. Do not call `malloc`, `calloc`, `realloc`, `free`, `sbrk` or `mmap`.
- Do not keep blocks or data in global or static arrays. A small array of free-list heads (Level 4, at most a few dozen pointers) is fine. Single global variables, like `heap_start`, are fine.
- Keep the block layout in `mm.c` as it is. Level 5's tests read it.

Why `mm_malloc` and not `malloc`

If your functions were called `malloc` and `free`, they would replace the C library's for the whole program. Then `printf`, the driver and the debugger would all run on your half-finished allocator, and one bug in it would break everything at once. With the `mm_` names, only the requests in the trace go to your code.

For the same reason, the driver does not use the real `sbrk`, which macOS no longer supports. The driver asks the C library for one 64 MB region when it starts, and `mem_sbrk(n)` hands it to your allocator a piece at a time, exactly the way `sbrk(n)` would: it returns the old end of the heap and moves the end `n` bytes further. Like `sbrk`, it returns `(void *)-1` when it fails, not `NULL`.

From the Slides to the Lab

Lecture 11 was written for this lab. It uses the same block layout, the same helper names and the same example heap, so its code is the lab's code:

Lecture 11 slide	In `mm.c`
"How Big Is a Block?"	`block_size` (Level 1)
"An Implicit Free List"	`find_fit` and the given `mm_malloc` (Level 1)
"free()"	`mm_free` (Level 1)
"Splitting a Free Block"	splitting in `place` (Level 2)
"Coalescing" and "The Four Cases"	`coalesce` (Level 2)
"An Explicit Free List"	Level 4
"A Heap Checker"	`mm_check` (Level 5)

The slides use `sbrk` when they talk about real allocators. The lab uses `mem_sbrk`, explained above.

The Block Layout

Every block has an 8-byte header, a payload, and an 8-byte footer. The payload is what `mm_malloc` returns, and its address must be a multiple of 16.

```
+----------+-----------------------------------+----------+
|  Header  |  payload (what mm_malloc returns)   |  Footer  |
|  8 bytes |  starts at a multiple of 16         |  8 bytes |
```

```
  +----------+-------------------------------------+----------+
  ^ h        ^ payload_of(h)                       ^ footer_of(h)
  |<---------------------- h->size ------------------------->|
```

```c
typedef struct header {
    uint32_t size;     /* the whole block in bytes: header + payload + footer */
    uint32_t is_free;  /* 1 if the block is free, 0 if it is allocated */
} Header;

typedef Header Footer; /* the footer is a copy of the header */
```

Three rules follow from this:

- **Block sizes are multiples of 16.** A request for `n` bytes needs a block of `n + 16` bytes (payload plus header and footer), rounded up to a multiple of 16. The smallest block is `MIN_BLOCK`, 32 bytes. So 16 bytes needs 32, 48 bytes needs 64, and 112 bytes needs 128. That is `block_size(n)`, the first function you write.
- **The heap starts with an 8-byte fence.** `mm_init` (given) puts it there. Because of it, every block starts 8 bytes past a multiple of 16, and every payload starts on a multiple of 16. The fence looks like the footer of an allocated block, so the first block never tries to merge with something before it.
- **The footer lets you go backwards.** The 8 bytes just before block `h` are the footer of the block before it. `prev_footer(h)->is_free` tells you whether that block is free, and `prev_block(h)` uses its size to find its header. This is what Lecture 11 calls a boundary tag.

```
heap+0  heap+8
+-------+---------+---------+-----+---------+
| fence | block   | block   | ... | block   |
+-------+---------+---------+-----+---------+
        ^ heap_start                        ^ heap_end
```

`mm.c` gives you these helpers. Each is one line, and you should use them rather than repeat the pointer arithmetic:

Helper	Returns
`round_up(n)`	`n` rounded up to a multiple of 16
`payload_of(h)`, `header_of(p)`	from a block to its payload, and back
`footer_of(h)`	block `h`'s footer
`next_block(h)`	the block after `h`. For the last block this is `heap_end`, so check `at_end`
`prev_footer(h)`	the footer of the block before `h` (the fence, for the first block)
`prev_block(h)`	the block before `h`. Use it only when `prev_footer(h)->is_free`
`at_end(h)`	true if `h` is at `heap_end`, so not a block
`set_block(h, size, is_free)`	writes `h`'s header **and** footer. Use it every time a block changes

Worked Example: The Lecture 11 Heap

The "Fragmentation" slide in Lecture 11 shows this heap:

```
[Alloc 32][Free 64][Alloc 32][Free 96]
```

and says a 128-byte request cannot be served from it, because 160 bytes are free but the largest free piece is 96. The first trace, `traces/t1-lecture.tr`, builds that heap and makes that request. Here it is without the trace's own comments. The notes after `#` are added here to show what each line means:

```
a 0 16        # p0 = mm_malloc(16)
a 1 48        # p1 = mm_malloc(48)
a 2 16
a 3 80
```

```
h 232         # check: the heap is at most 232 bytes
f 1           # mm_free(p1)
f 3
a 4 112
h 360
a 5 40
h 360
```

Work through it by hand first. Then compare with what the driver prints for a working level 1 allocator:

```
$ ./mdriver -v traces/t1-lecture.tr
    7  a 0 16               -> heap+16
    8  a 1 48               -> heap+48
    9  a 2 16               -> heap+112
   10  a 3 80               -> heap+144
   13  h 232                heap is 232 bytes, limit 232
   16  f 1
   17  f 3
   18  a 4 112              -> heap+240
   23  h 360                heap is 360 bytes, limit 360
   24  a 5 40               -> heap+48
   28  h 360                heap is 360 bytes, limit 360
ok:  8 requests, peak 184 bytes in use, heap 360 bytes, utilization 51.1%
```

The first column is the line number in the trace. Addresses are shown as offsets from the start of the heap.

1. **`a 0 16`.** `block_size(16)` is 16 + 16 = 32. There are no blocks yet, so `find_fit` finds nothing and `grow_heap(32)` calls `mem_sbrk(32)`. The new block starts at heap+8, right after the fence, and its payload at heap+16.
2. **`a 1 48`, `a 2 16`, `a 3 80`.** These need 64, 32 and 96 bytes. Each grows the heap again, so the blocks start at heap+40, heap+104 and heap+136, and the payloads at heap+48, heap+112 and heap+144: all multiples of 16. The heap is now 8 + 32 + 64 + 32 + 96 = 232 bytes, which the `h 232` line checks.
3. **`f 1`, `f 3`.** `mm_free` marks the 64- and 96-byte blocks free. This is the slide's heap. `./mdriver -v -d` prints it after each request:

   ```
   17  f 3
       heap: 232 bytes
       heap+8      alloc     32 bytes   payload at heap+16
       heap+40     free      64 bytes   payload at heap+48
       heap+104    alloc     32 bytes   payload at heap+112
       heap+136    free      96 bytes   payload at heap+144
   ```

4. **`a 4 112`.** `block_size(112)` is 128. `find_fit(128)` walks all four blocks: two are allocated, and the free ones are 64 and 96 bytes, both too small. So the heap grows by 128 bytes, the new block starts at heap+232, and its payload is at heap+240. This is the slide's external fragmentation: 160 bytes were free, just not in one piece.
5. **`a 5 40`.** `block_size(40)` is 40 + 16 = 56, rounded up to 64. `find_fit(64)` walks from the start: heap+8 is allocated, and heap+40 is free and 64 bytes. First fit takes it, and the payload is heap+48 again. The heap does not grow, and `h 360` checks that.

An allocator that never reuses a free block would put request 5 at the end of the heap. The heap would be 424 bytes and the last check fails:

```
FAIL  traces/t1-lecture.tr, line 28 (h 360):
      the heap is 424 bytes here, but it should be at most 360.
      A 40-byte request needs a 64-byte block. First fit finds the free
      64-byte block at heap+40 and reuses it, so the heap must not grow.
```

When an `h` check fails, the driver prints the comment written above it in the trace, which says what should have

happened.

What You Write

Level 1: implicit free list, first fit (60 pts)

`mm_malloc` is given. It calls four functions, and you write them, plus one line in `mm_free`:

Function	What it does
`block_size(n)`	the block size for an `n`-byte request: `n + 16`, rounded up to 16, at least `MIN_BLOCK`
`find_fit(size)`	first fit: walk the blocks from `heap_start` until `at_end`, and return the first free block of at least `size` bytes, or `NULL`. This is the loop on the "An Implicit Free List" slide
`grow_heap(size)`	call `mem_sbrk(size)`. If it fails, return `NULL`. Otherwise move `heap_end`, make the new bytes one **free** block of `size` bytes with `set_block`, and return it
`place(h, size)`	mark free block `h` allocated, keeping its size: `set_block(h, h->size, 0)`
`mm_free(p)`	mark the block free: `set_block(h, h->size, 1)`

That is all level 1 needs. Freed blocks are reused whole, and a block that is bigger than the request stays bigger.

Level 2: splitting and coalescing (15 pts)

- **Split, in `place`.** If the free block is bigger than the request by at least `MIN_BLOCK` bytes, cut it in two: the front becomes an allocated block of `size` bytes, and the rest becomes a new free block right after it. Otherwise hand out the whole block, as in level 1.
- **Coalesce, in `coalesce`.** `mm_free` calls `coalesce(h)` after marking `h` free. Merge `h` with the next block if that one is free, and with the previous block if that one is free. That makes four cases (neither, next only, previous only, both), and with the helpers each one is a line or two. When `h` merges into the block before it, the merged block starts at the previous block, so return that.

Do both. If blocks coalesce but never split, every freed region merges into one big block, the next small request takes all of it, and level 1's `t3-reuse.tr` starts failing.

Level 3: realloc and calloc (10 pts)

- `mm_realloc(p, n)` returns a block of at least `n` bytes holding the first `min(old size, n)` bytes of the old one. The simplest correct version calls `mm_malloc(n)`, copies with `memcpy`, and frees the old block. The old block's payload is `h->size - OVERHEAD` bytes. `mm_realloc(NULL, n)` is `mm_malloc(n)`, and `mm_realloc(p, 0)` frees `p` and returns `NULL`.
- `mm_calloc(nmemb, size)` returns `nmemb * size` bytes, all zero. Check for overflow **before** you multiply: if `nmemb != 0 && size > SIZE_MAX / nmemb`, the product does not fit in a `size_t`, so return `NULL`.

Level 4: an explicit free list (10 pts)

With an implicit list, `find_fit` walks past every allocated block to reach the free ones. Level 4's traces keep 20,000 to 30,000 blocks allocated, so that takes seconds. The limit is 0.2 seconds of CPU time per trace. A correct explicit list takes a few thousandths of a second, even on an old laptop.

Keep the free blocks on their own doubly linked list:

- A free block's payload is not in use, so store the list pointers there: `next` and `prev`, 16 bytes in all. `MIN_BLOCK` is 32 so that even the smallest block has room for them.
- Add a global `static Header *free_list;` and set it to `NULL` in `mm_init`. The driver calls `mm_init` more than once, so every global must be reset there.

- Between requests, every free block must be on the list and nothing else may be. One way to keep that true:
  - `find_fit` walks `free_list` instead of the heap, and takes the block it returns off the list.
  - `grow_heap`'s new block never goes on the list, because `place` allocates it straight away.
  - `place` puts the free half of a split on the list.
  - `coalesce` takes any free neighbour it merges with off the list, and puts the merged block on.
  - `mm_realloc`, if you make it split or merge, follows the same rules.
- Adding blocks at the front of the list (LIFO) is enough for level 4.

Level 5: the heap checker (5 pts)

`mm_check` walks the heap and returns 0 if everything is consistent. If not, it prints what is wrong to `stderr` and returns nonzero. `./mdriver -c` calls it after every request, which is the fastest way to find the request that first breaks your heap. Check at least:

1. every block's size is a multiple of 16 and at least `MIN_BLOCK`
2. every block's header and footer are equal
3. no two free blocks are next to each other (they should have been coalesced)
4. walking the blocks ends exactly at `heap_end`
5. if you did level 4: every block on the free list is free, and the number of blocks on the list equals the number of free blocks in the heap

Level 5 has five checks, 1 point each: `mm.c` compiles with no warnings under `-Wall -Wextra`; `mm_check` returns 0 on every request of six correct traces; and `mm_check` notices each of three heaps the driver breaks on purpose (`./mdriver -C 1`, `-C 2` and `-C 3`). Check your loop bounds: a checker that crashes on a broken heap has not noticed anything.

Bonus: use memory well (+10)

Trace	+5 if utilization reaches 80%
`b1-realloc.tr`	one block grows 64 bytes at a time. Grow it in place when you can: when the next block is free and big enough, or when the block is the last one in the heap and `mem_sbrk` can extend it
`b2-fit.tr`	first fit spoils big free blocks with small requests. Choose the best fit (the smallest free block that is big enough), or keep separate free lists for different sizes

The Driver and the Traces

A trace has one request per line:

Line	Means
`a id n`	`p[id] = mm_malloc(n)`
`f id`	`mm_free(p[id])`
`r id n`	`p[id] = mm_realloc(p[id], n)`
`c id k n`	`p[id] = mm_calloc(k, n)`
`h bytes`	a check: the heap is at most this many bytes here

After every request, the driver checks that the payload address is a multiple of 16, that the whole payload is inside the heap, and that it does not overlap any other allocated block. It fills every block it gets with a pattern, and when the block is freed or reallocated it checks that the pattern is still there. If your allocator writes a header, a footer or a list pointer into someone's payload, the driver finds it.

Utilization is the most payload the trace had allocated at any one time, divided by the final size of the heap. An allocator that never reuses anything scores very low. Headers, footers and rounding cost a little, so nobody reaches 100%. Some checks need a minimum utilization.

Options worth knowing:

Option	Does
`./mdriver -v trace`	prints every request and the address it got
`./mdriver -v -d trace`	also prints the heap after every request (`mm_print_heap`)
`./mdriver -c trace`	calls your `mm_check` after every request
`./mdriver -C 1`	heap checker test 1 (also 2 and 3)

If your code crashes, the driver catches it and tells you the line of the trace it was on.

Testing

`make test` runs each check and prints a score. Here is a student who has finished levels 1 and 2, whose `mm_realloc` forgets the `memcpy`:

```
Level 1: implicit free list, first fit (60 pts)
  PASS  L1  t1-lecture.tr  the Lecture 11 example
  PASS  L1  t2-sizes.tr    block sizes and alignment
  PASS  L1  t3-reuse.tr    free blocks are reused
  PASS  L1  t4-churn.tr    random, small sizes   (util 60.8%)

Level 2: splitting and coalescing (15 pts)
  PASS  L2  t5-split.tr    split a big free block
  PASS  L2  t6-coalesce.tr merge with next and previous
  PASS  L2  t7-mixed.tr    random, 8 bytes to 4 KB   (util 86.9%)

Level 3: realloc and calloc (10 pts)
  FAIL  L3  t8-realloc.tr  realloc keeps the data
        traces/t8-realloc.tr, line 8 (r 0 300):
              block 0's data changed in mm_realloc:
              byte 0 (at heap+208) is 0xa5, but the driver wrote 0xae there.
              mm_realloc must copy the old data (the first min(old size, new size) bytes)
              into the block it returns.
  PASS  L3  t9-calloc.tr   calloc zeroes, and overflow

Level 4: an explicit free list (10 pts)
  FAIL  L4  t10-many.tr    30,000 blocks, fast
        traces/t10-many.tr: too slow: the requests took more than 0.20 s of CPU time.
              Walking past every allocated block on every request is too slow here.
  ...
------------------------------------------------
Level 1: 4/4 checks   60/60 pts
Level 2: 3/3 checks   15/15 pts
Level 3: 1/2 checks    5/10 pts
Level 4: 0/2 checks    0/10 pts
Level 5: 1/5 checks    1/5 pts
Bonus:   0/2 checks   +0/10
Score: 81/100, bonus +0
------------------------------------------------
```

Each trace targets one thing, so a failure tells you which part is wrong:

Trace	What it is for
`t1-lecture.tr`	the Lecture 11 example above
`t2-sizes.tr`	`block_size` and alignment: 326 requests, no frees, starting with the sizes where rounding goes wrong (15, 16, 17, 31, 32, 33, …)
`t3-reuse.tr`	free blocks and ask for the same sizes again: the heap must not grow
`t4-churn.tr`	3,000 random requests from seven small sizes, needs 40% utilization
`t5-split.tr`	sixty small requests must fit in one freed 4,000-byte block
`t6-coalesce.tr`	free fifty neighbours last-to-first, then first-to-last, and ask for one block as big as all of them. The first half needs merging with the next block, the second half with the previous block
`t7-mixed.tr`	4,000 random requests from 8 bytes to 4 KB, needs 55% utilization
`t8-realloc.tr`	grow, shrink and move blocks with `mm_realloc`, needs 30% utilization
`t9-calloc.tr`	zeroed memory, even when the block is reused, and `NULL` for requests that overflow or are bigger than the heap
`t10-many.tr`, `t11-waves.tr`	tens of thousands of blocks: 0.2 s of CPU time and 65% utilization
`b1-realloc.tr`, `b2-fit.tr`	the bonus

Two notes:

- The level 5 "no warnings" point and the bonus count only once every level 1 check passes, so the starter code scores 0.
- Your `mm.c` is also run on traces you have not seen, of the same kinds. An allocator that works on these traces only by luck will lose points there.

Grading

Level	Requirement	Points
1	**Implicit free list, first fit:** `t1` to `t4`	60
2	**Split and coalesce:** `t5` to `t7`	15
3	**`mm_realloc` and `mm_calloc`:** `t8`, `t9`	10
4	**Explicit free list:** `t10`, `t11` within 0.2 s	10
5	**Heap checker and a clean build:** five checks, 1 point each	5
Bonus	`b1` and `b2` reach 80% utilization, 5 points each	+10

Level 1 alone is a passing grade on this lab. Do the levels in order: each one builds on the code before it.

Hints

- **Draw the heap.** Before you write `coalesce`, draw three blocks on paper and work out which header and footer change in each of the four cases. Then check your drawing with `./mdriver -v -d traces/t6-coalesce.tr`.
- **Call `set_block` every time a block changes.** Most heap bugs in this lab are a header and footer that disagree. `set_block` writes both.
- **Never read past the last block.** `next_block` of the last block is `heap_end`, which is not a block. Check `at_end` before you look at `next->is_free`. If `mm_free` crashes on the last block of a trace, this is why.
- **Find the first bad request with `-c`.** Once your `mm_check` works, run `./mdriver -c` on the failing trace. It stops at the first request after which the heap is inconsistent, which is usually much earlier than the request that crashes.
- **`grow_heap` returns a free block** and `place` then allocates it, so that `mm_malloc` handles new and reused blocks the same way.
- **Optional, for utilization:** if the last block in the heap is free when nothing fits, you only need `mem_sbrk` for the difference. Grow that block instead of adding a new one.

- **Debug with `-O0` if you use a debugger.** The Makefile builds with `-O2` for level 4's timing. For `gdb` or `lldb`, rebuild with `make clean && make CFLAGS="-std=c17 -Wall -Wextra -O0 -g"`.

Submission

Upload `mm.c` to the Lab 3 assignment on **Brightspace**. Submit the source file itself: not a screenshot, not a zip, not a link to a repository. Put your name at the top, and replace the design paragraph at the top with a few sentences about your own allocator.

Your `mm.c` must compile with the flags in the Makefile, against the `mdriver.c`, `memlib.c` and headers in the kit, which you must not change. Code that does not build earns no points, so run `make test` one last time before you upload.

References

- CS:APP §9.9 (Dynamic Memory Allocation). §9.9.12 builds an implicit-list allocator like level 1 and 2. Its block layout is different: it packs the allocated flag into the low bit of the size and uses a prologue and an epilogue block. Its code will not work here as it is, but the ideas are the same.
- Lecture 11, "Kernel III: Memory Allocator Design"
- Dan Luu, Malloc tutorial